

PHPMySQL UNIT - 1

LINK:-<https://docs.google.com/document/d/1awb9Oe-QmYohVyQJeUrfZ6RVFaXdYU57OZAs2RF1gLE/edit?usp=sharing>

Section 1: Introduction to PHP

1.1. What is PHP?

PHP, a recursive acronym for "PHP: Hypertext Preprocessor," is a widely-used, open-source, server-side scripting language. Its primary design purpose is for web development, enabling the creation of dynamic and interactive web pages.¹ Unlike client-side languages such as JavaScript, which execute in the user's web browser, PHP code is processed on the web server. This server-side execution model is fundamental to its power and security, as the underlying business logic and data access remain hidden from the end-user.

Core Characteristics

- **Server-Side Scripting:** When a user requests a .php file, the web server passes the request to the PHP interpreter. The interpreter executes the PHP code, which may involve tasks like querying a database, processing form data, or manipulating files. The final output of this process is typically a standard HTML document, which is then sent to the user's browser for rendering. This ensures that the user only sees the result, not the source code that produced it.¹
- **HTML-Embedded:** PHP code can be seamlessly embedded within HTML documents. A PHP file can contain a mixture of HTML tags and PHP script blocks. The PHP interpreter only processes the code found within its designated tags, while all surrounding HTML is passed directly to the output. This integration makes it exceptionally convenient to inject dynamic content into static page structures.¹
- **Cross-Platform Compatibility:** PHP is designed to be platform-independent. A PHP application developed on a Linux server can typically be deployed on a Windows or macOS server with minimal to no modification. This flexibility makes it a versatile choice for developers working in diverse environments.¹

1.2. Evaluation and History of PHP

The evolution of PHP from a small personal project into a dominant force on the web is a testament to its utility and the strength of its open-source community.

- **Origins (1994-1995):** The language was created by Rasmus Lerdorf in 1994. It began not as a language, but as a collection of Common Gateway Interface (CGI) binaries written in C, which he called "Personal Home Page Tools." These tools were used to track visitors to his online resume. Over time, Lerdorf expanded the toolset, creating "PHP/FI" (Personal Home Page/Forms Interpreter), which included more features for handling web forms and database interaction.¹ The name "PHP/FI" clearly reflects these utilitarian origins, emphasizing its role as a practical tool for form interpretation rather than a general-purpose language.
- **PHP 3 (1998):** This version marked a pivotal moment in PHP's history. Two developers, Zeev Suraski and Andi Gutmans, found PHP/FI inadequate for an e-commerce application they were developing. They undertook a complete rewrite of the underlying parser, creating the **Zend Engine** (a portmanteau of their first names, Zeev and Andi). The release of PHP 3 with the new engine provided significantly better performance, a more consistent syntax, and extended functionality, including basic object-oriented support. This transition from a personal project to a community-driven, architecturally sound language was crucial for its widespread adoption.¹
- **PHP 4 (2000):** Built upon the improved Zend Engine, PHP 4 focused on enhancing performance and reliability. It introduced features like output buffering, more robust session handling, and a wider array of built-in functions, making it suitable for more complex and high-traffic websites.
- **PHP 5 (2004):** This was another major architectural overhaul, driven by the need to compete with more structured languages like Java and ASP.NET. Powered by the new Zend Engine 2, PHP 5 introduced a fully revamped Object-Oriented Programming (OOP) model with features like public, private, and protected visibility, abstract classes, and interfaces. It also brought significant improvements to XML handling through the SimpleXML extension and introduced robust error handling with try/catch exception blocks. This version solidified PHP's position as a mature language capable of powering large-scale, enterprise-level applications.¹

This evolutionary path highlights a core characteristic of PHP: it balances its legacy of simplicity and rapid development with the sophisticated features required by modern software engineering.

Section 2: Basic Syntax

2.1. PHP Tags

To embed PHP code within a file, it must be enclosed in special tags that signal the start and end of a script block to the PHP interpreter.

- **Standard Tags:** The universally accepted and recommended method is to use the `<?php...?>` tags. This syntax is always available, regardless of server configuration, and ensures maximum portability.¹
- **Short Tags:** An alternative is the short-echo tag `<?=...?>`, which is a shorthand for `<?php echo... ;?>`. Another short tag, `<?...?>`, also exists. However, its use is discouraged because it can be disabled in the `php.ini` configuration file via the `short_open_tag` directive. Relying on it can make code non-portable. Furthermore, it can conflict with XML processing instructions, which use the same syntax.¹
- **Script Tags:** The `<script language="php">... </script>` syntax is also valid but is rarely used in modern practice.¹

2.2. Embedding PHP in HTML

PHP's design allows it to be seamlessly mixed with HTML. The PHP parser scans a file and executes only the code found within the designated PHP tags, passing any text outside these tags directly to the output stream.

Code Snippet:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP and HTML</title>
</head>
<body>
  <h1>User Profile</h1>
  <p>Welcome, <?php echo "John Doe";?>!</p>
  <p>Your membership expires on: <?php echo date('F j, Y', strtotime('+1 month'));?>.</p>
</body>
</html>
```

In this example, the PHP code dynamically inserts a name and calculates a future date, embedding them directly into the static HTML structure.¹

2.3. Statements and Semicolons

A statement in PHP is a single instruction, such as assigning a value to a variable or calling a function. Each simple statement must be terminated with a semicolon (;). The semicolon acts as a separator, allowing the parser to distinguish where one instruction ends and the next begins. Omitting it will result in a syntax error.¹

The closing PHP tag `?>` implicitly contains a semicolon. Therefore, the semicolon on the very last statement within a PHP block is technically optional. However, it is a widely followed best practice to always include it to prevent errors when adding more code later.¹

Code Snippet:

PHP

```
<?php
    echo "This is the first statement.";
    $variable = 10 + 5; // Semicolon is mandatory here.
    echo $variable      // Semicolon is optional here before the closing tag.
?>
```

2.4. Comments in PHP

Comments are non-executable parts of a script that are ignored by the PHP interpreter. They are essential for documenting code, explaining complex logic, and making the code more understandable for other developers (or for oneself in the future).

- **Single-line Comments:** PHP supports two styles for single-line comments. Everything from the comment marker to the end of the line is ignored.
 - C++ Style: `//`
 - Shell Style: `#`
- **Multi-line (Block) Comments:** These comments start with `/*` and end with `*/`. They can span multiple lines and are ideal for longer descriptions or for temporarily disabling a block of code during debugging. It is important to note that block comments in PHP cannot be nested.¹

Code Snippet:

PHP

```
<?php
// This is a single-line comment used to explain the next line.
$username = "admin";

# This is another style of single-line comment.
echo "Hello, ". $username; // A comment can also follow a statement.

/*
This is a multi-line comment block.
It is useful for providing detailed explanations of a function
or a complex section of code.
*/
$password = "12345";
?>
```

Section 3: Defining Variables and Constants

3.1. PHP Variables

A variable is a symbolic name associated with a value that can be changed during the execution of a script. It acts as a container for storing information.

- **Syntax and Naming Rules:**
 - All variables in PHP are denoted by a leading dollar sign (\$).¹
 - The name following the \$ must start with a letter or an underscore (_).
 - The name cannot begin with a number.
 - Subsequent characters can be letters, numbers, or underscores.
 - Variable names are **case-sensitive**. This means \$name, \$Name, and \$NAME are three distinct variables.¹
- **Assignment:** The assignment operator (=) is used to assign a value to a variable. As PHP is a loosely typed language, there is no need to declare the data type of a variable before assignment; the type is inferred from the value at runtime.¹

Code Snippet:

PHP

```
<?php
$site_name = "Tech University"; // A string variable
$student_count = 2500; // An integer variable
$_course_fee = 499.99; // A float variable

echo "Welcome to ". $site_name;
echo "We have ". $student_count. " students.";
?>
```

3.2. PHP Constants

A constant is an identifier for a value that, once defined, cannot be changed or undefined during the script's execution. Constants are ideal for storing values that should remain static, such as configuration settings.

- **Defining Constants:** Constants are defined using the `define()` function. This function requires at least two arguments: the name of the constant and its value.¹
- **Naming Rules:**
 - Constant names do not have a `$` prefix.
 - By convention, constant names are written in all uppercase letters to distinguish them from variables (e.g., `DATABASE_HOST`).
 - The naming rules are the same as for variables (must start with a letter or underscore).

Code Snippet:

PHP

```
<?php
define("MAX_LOGIN_ATTEMPTS", 5);
define("ADMIN_EMAIL", "admin@example.com");

echo "You have a maximum of ". MAX_LOGIN_ATTEMPTS. " login attempts.";
?>
```

3.3. Tabular Difference: Variables vs. Constants

The distinction between variables and constants is fundamental to structuring PHP applications correctly.

Feature	Variables	Constants
Syntax	Start with a \$ prefix (e.g., \$myVar).	Defined with define("MY_CONST", value);. No prefix.
Re-assignability	Can be reassigned a new value at any time.	Cannot be changed or undefined after being set.
Scope	Scope is typically local to a function, but can be made global or static.	Automatically global and can be accessed from anywhere in the script.
Data Types	Can hold any data type (scalar, compound, resource, etc.).	Traditionally hold only scalar values (integer, float, string, boolean).
Case-Sensitivity	Case-sensitive (e.g., \$name is different from \$NAME).	Case-sensitive by default. Convention is to use uppercase.
Determination	Value is determined and can be changed at runtime.	Value is typically set at compile time and is fixed.

The syntactic difference between variables (\$) and constants (no prefix) is a deliberate design choice that aids the PHP parser. When the parser encounters a \$ token, it recognizes a variable whose value is dynamic. When it encounters an identifier without a \$, it can immediately look it up in the static constant table. This distinction is particularly important for variable interpolation within double-quoted strings. The parser specifically searches for the \$ symbol to identify variables that need to be replaced with their values, while leaving constant names as literal text. This design reinforces the conceptual separation: variables are for dynamic, runtime data, while constants are for static, configuration-like values that are known before execution.

Section 4: PHP Data Types

4.1. Introduction to Data Types

PHP is a **dynamically-typed language**, often referred to as loosely-typed. This means that variables are not bound to a specific data type. The type of a variable is determined at runtime by the value it holds. This provides flexibility but requires careful management to avoid unexpected behavior.⁶

4.2. Scalar Types (Single Value)

Scalar types represent a single value.

- **Integer:** A whole number (without a decimal point), which can be positive or negative. Integers can be specified in several notations: decimal (base 10), hexadecimal (prefixed with 0x), octal (prefixed with 0o), or binary (prefixed with 0b).⁸
 - **Code Snippet:** `$user_id = 101;`
- **Float (or Double):** A number containing a decimal point or represented in exponential (scientific) notation. These are also known as floating-point numbers or real numbers.⁸
 - **Code Snippet:** `$item_price = 39.95; $avogadro_number = 6.022e23;`
- **String:** A sequence of characters enclosed in quotes. Strings can contain letters, numbers, symbols, and whitespace.⁵
 - **Code Snippet:** `$course_name = "PHP Programming";`
- **Boolean:** The simplest type, representing two possible states: TRUE or FALSE. Boolean values are case-insensitive. They are fundamental to all conditional logic.⁵
 - **Code Snippet:** `$is_active = true;`

4.3. Compound Types (Multiple Values)

Compound types can hold multiple values.

- **Array:** An ordered map that associates values with keys. Keys can be integers (for indexed arrays) or strings (for associative arrays). Arrays are one of PHP's most versatile data structures.¹
 - **Code Snippet:** `$student_grades = array("Math" => 95, "Science" => 88);`
- **Object:** An instance of a programmer-defined class. Objects bundle data (properties) and the functions that operate on that data (methods) into a single unit.¹
 - **Code Snippet:** `class Student { ... } $new_student = new Student();`

4.4. Special Types

- **NULL:** A special type that has only one value: NULL. A variable of type NULL is a variable that has no value assigned to it. It is also the value of a variable that has been explicitly unset().⁶
 - **Code Snippet:** `$middle_name = NULL;`
- **Resource:** A special variable holding a reference to an external resource, such as a database connection, a file handle, or an image canvas. These are created and used by specific functions.⁶
 - **Code Snippet:** `$db_connection = mysqli_connect("localhost", "user", "pass", "db");`

4.5. Type Juggling and Casting

- **Type Juggling (Implicit Conversion):** This is PHP's automatic conversion of a value from one data type to another based on the context of an operation. For example, if a numeric string is used in a mathematical calculation, PHP will automatically convert it to a number.¹¹
 - **Code Snippet:** `$total = "10" + 5; // PHP converts "10" to an integer. $total becomes 15.`
- **Type Casting (Explicit Conversion):** This is the manual conversion of a value to a specific data type. It is done by prefixing the value with the desired type in parentheses. This gives the developer explicit control over the data type.¹
 - **Allowed Casts:** (int), (bool), (float), (string), (array), (object).
 - **Code Snippet:** `$price_string = "199.99"; $price_float = (float) $price_string;`

Type juggling is a core design philosophy of PHP, intended to make the language more forgiving and accessible. While this automatic conversion reduces code verbosity, it can also be a significant source of subtle bugs and security vulnerabilities, especially when combined with loose comparison operators (==). For instance, the string "10 apples" might be juggled to the integer 10 in a numeric context, which could lead to unexpected logical outcomes. The language provides strict comparison operators (===) and explicit type casting as countermeasures, allowing developers to enforce type safety when precision and predictability are required.

Section 5: Operator and Expression

5.1. Expressions

An expression is a fundamental building block of PHP, best defined as "anything that has a value".¹⁴ The simplest expressions are literal values (like 5 or "hello") and variables (like \$x). More complex expressions are formed by combining these with operators (e.g., \$x + 5). In PHP, nearly every construct is an expression. For example, an assignment like \$a = 5 not only assigns the value 5 to \$a but the entire assignment itself evaluates to the value 5, allowing for chained assignments like \$b = \$a = 5;¹⁴

5.2. Arithmetic Operators

These operators are used to perform standard mathematical calculations.¹⁵

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- % (Modulus): Returns the remainder of a division.
- ** (Exponentiation): Raises the left operand to the power of the right operand.

Code Snippet:

PHP

```
$a = 10;  
$b = 3;  
$remainder = $a % $b; // $remainder will be 1  
$power = 2 ** 3;    // $power will be 8
```

5.3. Assignment Operators

These operators are used to assign values to variables. The basic assignment operator is =.¹⁵

- **Combined Assignment Operators:** These provide a shorthand for performing an operation on a variable and then assigning the new value back to it.
 - += (Add and assign)
 - -= (Subtract and assign)
 - *= (Multiply and assign)
 - /= (Divide and assign)
 - %= (Modulus and assign)
 - .= (Concatenate and assign)

Code Snippet:

PHP

```
$counter = 10;  
$counter += 5; // Equivalent to $counter = $counter + 5; $counter is now 15.
```

```
$message = "Hello";  
$message.= ", World!"; // $message is now "Hello, World!".
```

5.4. Comparison Operators

Comparison operators are used to compare two values, and the expression evaluates to a boolean value (TRUE or FALSE).¹⁵

- == (Equal): Returns TRUE if the values are equal after type juggling.
- === (Identical): Returns TRUE only if the values are equal AND they are of the same data type.
- != or <> (Not equal): Returns TRUE if the values are not equal after type juggling.
- !== (Not identical): Returns TRUE if the values are not equal OR they are not of the same data type.
- < (Less than)
- > (Greater than)
- <= (Less than or equal to)
- >= (Greater than or equal to)
- <=> (Spaceship operator, available in PHP 7+): Returns -1 if the left operand is less than the right, 0 if they are equal, and 1 if the left is greater than the right.

5.5. Tabular Difference: == (Equal) vs. === (Identical)

Understanding the difference between loose (==) and strict (===) comparison is critical for writing robust and secure PHP code. The following table illustrates how type juggling affects the outcome of loose comparisons.

Expression	== Result	=== Result	Explanation
5 == "5"	true	false	== converts the string "5" to an integer before comparing values. === fails because the types (integer and string) are different.
0 == false	true	false	== considers false to be equivalent to 0. === fails because the types (integer and boolean) are different.
0 == ""	true	false	== converts an empty string to 0 in a numeric comparison. === fails due to different types.
0 == NULL	true	false	== considers NULL to be equivalent to 0. === fails due to different types.
false == NULL	true	false	Both are considered "falsy" and are loosely equal. === fails due to different types.
"" == NULL	true	false	Both are considered "empty" and are loosely equal. === fails due to different types.

5.6. Logical Operators

Logical operators are used to combine multiple conditional expressions.¹⁵

- **and** or **&&** (And): Returns TRUE if both operands are true.
- **or** or **||** (Or): Returns TRUE if either operand is true.
- **xor** (Exclusive Or): Returns TRUE if one operand is true, but not both.
- **!** (Not): Inverts the boolean value of its operand; returns TRUE if the operand is false.

The text-based operators (**and**, **or**) have a lower precedence than their symbolic counterparts (**&&**, **||**). This can lead to unexpected behavior in assignments. For example, `$result = true && false;` correctly assigns false to `$result`. However, `$result = true and false;` is evaluated as `($result = true) and false;`, which assigns true to `$result` before the **and** operation is evaluated. Due to this potential for confusion, it is a common best practice to exclusively use the **&&** and **||** operators.

5.7. Increment/Decrement Operators

These operators provide a concise way to increase or decrease a variable's value by one.¹⁵

- **Pre-increment (++\$var)**: Increments the variable's value by one, and then returns the *new* value.
- **Post-increment (\$var++)**: Returns the *original* value of the variable, and then increments its value by one.
- **Pre-decrement (--\$var)**: Decrements the variable's value by one, and then returns the *new* value.
- **Post-decrement (\$var--)**: Returns the *original* value of the variable, and then decrements its value by one.

Code Snippet:

PHP

```
$a = 10;  
$b = ++$a; // $a becomes 11, then $b is set to 11. Both are 11.
```

```
$c = 10;  
$d = $c++; // $d is set to the original value of $c (10), then $c becomes 11.
```

5.8. String and Array Operators

- **String Concatenation Operator (.)**: This operator is used to join two or more strings together.
 - **Code Snippet**: `$fullName = "John". " ". "Doe";`
- **Array Union Operator (+)**: This operator appends elements from the array on the right to the array on the left. It only adds elements for keys that do not already exist in the left-hand array; it does not overwrite existing values.

5.9. Operator Precedence

Operator	Description	Example	Result
<code>&&</code>	And	<code>1 == 1 && 2 == 2</code>	TRUE
<code>and</code>	And	<code>1 == 1 and 2 == 2</code>	TRUE
<code> </code>	Or	<code>1 == 1 2 == 3</code>	TRUE
<code>or</code>	Or	<code>1 == 1 or 2 == 3</code>	TRUE
<code>xor</code>	Exclusive Or	<code>1 == 1 xor 2 == 2</code>	FALSE
<code>!</code>	Not	<code>!(1 == 1)</code>	FALSE

Operator precedence defines the order in which operations are evaluated in a complex expression. For example, multiplication (*) has a higher precedence than addition (+), so `3 + 5 * 2` is evaluated as `3 + 10`, resulting in 13.¹⁸ Parentheses () can be used to override the default precedence and force a specific order of evaluation. For example, `(3 + 5) * 2` evaluates to `8 * 2`, resulting in 16.

Section 6: Decisions and Loops

6.1. Decision-Making Structures

Decision-making structures, or conditional statements, allow a program to execute different blocks of code based on whether certain conditions are met.

- **if Statement:** Executes a block of code only if the specified condition evaluates to TRUE.¹⁹
- **if-else Statement:** Provides an alternative block of code to execute if the if condition evaluates to FALSE.¹⁹
- **if-elseif-else Statement:** Allows for checking a sequence of multiple conditions. The first elseif block whose condition is TRUE will be executed. If none are true, the final else block (if present) is executed.¹⁹
- **switch Statement:** A more structured alternative to a long if-elseif-else chain. It evaluates a single expression and compares its value against a series of case labels. Execution jumps to the matching case. The break statement is essential to prevent "fall-through," where execution would continue into the next case block. A default case can be used to handle any value that doesn't match a specific case.¹⁹

Code Snippet (switch):

PHP

```
$grade = 'B';
switch ($grade) {
    case 'A':
        echo "Excellent!";
        break;
    case 'B':
        echo "Good job!";
        break;
    case 'C':
        echo "Fair.";
        break;
    default:
        echo "Please see your instructor.";
}
// Output: Good job!
```

6.2. Looping Structures

Loops are used to execute a block of code repeatedly until a certain condition is met.

- **while Loop:** The condition is evaluated *before* each iteration. The loop continues as long as the condition is TRUE. If the condition is initially FALSE, the loop body will never execute.²⁴
- **do-while Loop:** The condition is evaluated *after* each iteration. This guarantees that the loop body will execute at least once, even if the condition is initially FALSE.²⁴
- **for Loop:** Ideal for situations where the number of iterations is known beforehand. It consists of three parts separated by semicolons:
 1. **Initialization:** Executed once at the beginning of the loop.
 2. **Condition:** Checked before each iteration.
 3. **Increment/Decrement:** Executed at the end of each iteration.²⁴
- **foreach Loop:** Provides the simplest way to iterate over the elements of an array or an object. In each iteration, the value of the current element is assigned to a variable.²⁴

Code Snippet (foreach):

PHP

```
$courses =;  
echo "Available courses: ";  
foreach ($courses as $course) {  
    echo $course. " ";  
}  
// Output: Available courses: PHP MySQL HTML
```

6.3. Interrupting Loops: break and continue

- **break:** This statement immediately terminates the execution of the current for, foreach, while, do-while, or switch structure.²⁸
- **continue:** This statement skips the remainder of the current loop iteration and proceeds to the condition evaluation for the next iteration.³⁰

6.4. Mixing Decisions and Looping with HTML

PHP's design makes it easy to generate dynamic HTML content. Loops can be used to create

table rows, list items, or other repeating elements, while conditional statements can be used to selectively display content.

- **Alternative Syntax:** To improve readability when embedding control structures within HTML, PHP offers an alternative syntax that replaces the opening brace { with a colon : and the closing brace } with a corresponding endif;, endwhile;, endfor;, or endforeach; statement. This makes the code structure clearer in files with a lot of markup.³

Code Snippet (Alternative Syntax):

HTML

```
<h3>Student List</h3>
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    <?php foreach ($students as $student):?>
      <tr>
        <td><?php echo $student['name'];?></td>
        <td>
          <?php if ($student['is_active']):?>
            Active
          <?php else:?>
            Inactive
          <?php endif;?>
        </td>
      </tr>
    <?php endforeach;?>
  </tbody>
</table>
```

This alternative syntax is a direct acknowledgment by the language designers of PHP's primary use case in templating. It mitigates the "spaghetti code" problem that can arise from mixing logic and presentation by making the boundaries of control blocks more explicit and readable within an HTML context. This feature represents a step toward cleaner code architecture, predating the widespread adoption of dedicated templating engines.

Section 7: Function

7.1. What is a Function?

A function is a named, reusable block of code designed to perform a specific task. Functions are a cornerstone of structured programming, as they promote code organization, reduce redundancy (following the "Don't Repeat Yourself" or DRY principle), and simplify maintenance and debugging.³³

7.2. Defining and Calling a Function

- **Defining a Function:** A function is defined using the function keyword, followed by a unique function name, a set of parentheses (), and a block of code enclosed in curly braces {}. ³⁵
- **Calling a Function:** To execute the code within a function, you "call" it by writing its name followed by parentheses. ³³

Code Snippet:

PHP

```
// Define the function
function displayGreeting() {
    echo "Welcome to the system!";
}
```

```
// Call the function
displayGreeting(); // Outputs: Welcome to the system!
```

7.3. Function Parameters and Arguments

CALL BY VALUE	CALL BY REFERENCE
CALLING FUNCTION SENDS COPIES TO DATA.	CALLING FUNCTION SENDS ADDRESS OF DATA.
THE FORMAL PARAMETERS ARE ORDINARY VARIABLES.	THE FORMAL PARAMETERS ARE POINTER VARIABLE.
ATMOST ONLY ONE VALUES CAN BE SENT BACK TO THE CALLING FUNCTION.	SEVERAL RESULTS CAN BE SENT BACK TO THE CALLING FUNCTION.
ACTUAL PARAMETERS ARE AFFECTED BY CHANGES MADE WITHIN THE FUNCTION.	DIRECT CHANGES ARE MADE TO THE ACTUAL PARAMETERS.

- **Parameters and Arguments:**
 - **Parameters** are the variables listed inside the parentheses in the function definition. They act as placeholders for the values that will be passed into the function.
 - **Arguments** are the actual values or variables that are passed to the function when it is called.
- **Default Argument Values:** A parameter can be made optional by assigning it a default value in the function definition. If an argument for that parameter is not provided during the function call, the default value will be used.³⁷
 - **Code Snippet:** `function setPermissions($user, $level = 'read') { ... }`
- **Call by Value (Default Behavior):** By default, PHP passes arguments to functions by value. This means a *copy* of the argument's value is passed to the function. Any modifications to the parameter inside the function do not affect the original variable outside of it.¹
- **Call by Reference:** To allow a function to modify its arguments, you must pass them by reference. This is done by prefixing the parameter name in the function definition with an ampersand (&). This passes a reference to the original variable's memory location, so any changes made inside the function will directly affect the original variable.¹

7.4. Tabular Difference: Call by Value vs. Call by Reference

Feature	Call by Value	Call by Reference
Mechanism	A copy of the variable's value is passed to the function.	A reference (memory address) to the original variable is passed.
Syntax	<code>function myFunc(\$param) {...}</code>	<code>function myFunc(&\$param) {...}</code> (Note the & in the definition)
Effect on Original Variable	The original variable remains unchanged by the function.	The original variable is modified if the parameter is changed inside the function.
Performance	Can be less efficient for large data structures (arrays, objects) due to the overhead of copying data.	More efficient for large data structures as only a memory reference is passed.
Use Case	Ideal for when a function needs to use a value for calculations without altering the original source.	Necessary when a function's purpose is to modify its input variables (e.g., sorting an array in place).
Example	<code>\$a=10; func(\$a); // \$a is still 10</code>	<code>\$a=10; func(&\$a); // \$a is modified by the function</code>

7.5. Recursive Function

A recursive function is a function that calls itself, either directly or indirectly. This technique is powerful for solving problems that can be broken down into smaller, self-similar sub-problems. A crucial component of any recursive function is a **base case**: a condition that stops the recursion and prevents an infinite loop, which would otherwise lead to a stack overflow error.³⁶

Classic Example (Factorial): The factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n . It can be defined recursively as $n! = n * (n-1)!$, with the base case $0! = 1$.

Code Snippet:

PHP

```
function factorial($n) {  
    // Base case: The factorial of 0 or 1 is 1. This stops the recursion.  
    if ($n <= 1) {  
        return 1;  
    } else {  
        // Recursive step: The function calls itself with a smaller value (n - 1).  
        return $n * factorial($n - 1);  
    }  
}
```

```
echo factorial(5); // Outputs: 120 (since 5 * 4 * 3 * 2 * 1 = 120)
```

43

Section 8: String Manipulation

8.1. Creating and Accessing Strings

PHP provides several ways to define strings, each with different behaviors regarding variable parsing and escape sequence handling.

- **Single Quotes (')**: This is the simplest way to create a string. The content is treated literally. Variables and most escape sequences (like `\n`) are not processed. The only recognized escape sequences are `\'` (for a literal single quote) and `\\` (for a literal backslash). This method is slightly faster for static strings.⁴⁷
- **Double Quotes (")**: These strings are parsed by PHP. The parser looks for variables (e.g., `$name`) and replaces them with their values, a process known as **variable interpolation**. It also processes a wide range of escape sequences, such as `\n` (newline), `\t` (tab), and `\$` (literal dollar sign).⁴⁷
- **Heredoc (<<<)**: This syntax is designed for creating multi-line strings without the need to escape quotes. A heredoc block starts with `<<<` followed by an identifier, a newline, the string content, and finally the same identifier on a new line to close it. Heredocs behave like double-quoted strings, meaning variables and escape sequences are processed.⁴⁸
- **Accessing Characters**: Individual characters within a string can be accessed using zero-based array-like index notation with square brackets.¹ For example, `$myString[0]` returns the first character of `$myString`.

8.2. Tabular Difference: String Creation Methods

Feature	Single Quotes ('...')	Double Quotes ("...")	Heredoc (<<<ID...ID;)
Variable Interpolation	No (\$var is printed literally)	Yes (\$var is replaced by its value)	Yes
Escape Sequences	Only \' and \\ are recognized.	Many sequences like \n, \t, \\$ are processed.	Same as double quotes.
Multi-line Support	No (requires concatenation for multiple lines).	Yes (the string can span multiple lines in the code).	Yes (this is its primary use case).
Quote Escaping	Must escape single quotes (\').	Must escape double quotes (\").	No need to escape single or double quotes.

8.3. Searching & Replacing String

PHP offers a rich set of functions for finding and modifying substrings.

- **strpos(\$haystack, \$needle):** Finds the numeric position of the *first* occurrence of \$needle within \$haystack. It returns false if the substring is not found. A critical point is that if the substring is found at the very beginning, it returns 0. Since 0 is a "falsy" value in PHP, a loose comparison (== false) would incorrectly evaluate to true. Therefore, it is essential to use the strict identity operator (=== false or !== false) to check the return value.⁵¹
- **strstr(\$haystack, \$needle):** Finds the first occurrence of \$needle in \$haystack and returns the portion of the haystack from that point to the end. strchr() is an alias for this function.⁵³
- **str_replace(\$search, \$replace, \$subject):** Replaces all occurrences of the \$search string with the \$replace string within the \$subject. This function is case-sensitive and is highly optimized for simple, direct replacements.⁵⁴
- **str_ireplace():** A case-insensitive version of str_replace().

8.4. Formatting and Library Functions

Beyond searching and replacing, PHP provides a vast library of functions for string manipulation.

- **printf(\$format,...\$values):** Outputs a string formatted according to a specified

template. It uses placeholders like %s for a string, %d for a decimal integer, and %f for a float.⁵⁶

- **trim(\$string), ltrim(), rtrim():** Remove whitespace (or other specified characters) from the beginning and/or end of a string.⁵⁸
- **strlen(\$string):** Returns the length of a string in bytes. Note that for multi-byte character encodings like UTF-8, this may not correspond to the number of characters.
- **strtoupper(\$string), strtolower(\$string):** Convert a string to all uppercase or all lowercase, respectively.⁵⁹
- **ucfirst(\$string), ucwords(\$string):** Convert the first character of a string to uppercase, or the first character of each word in a string to uppercase.⁶⁰
- **explode(\$delimiter, \$string):** Splits a string into an array of substrings based on a specified delimiter.⁵³
- **implode(\$glue, \$array):** Joins the elements of an array into a single string, with each element separated by the specified glue string.⁵³

Code Snippets:

PHP

```
// strlen()
$greeting = "Hello";
echo strlen($greeting); // Outputs: 5
```

```
// strtoupper()
echo strtoupper($greeting); // Outputs: HELLO
```

```
// str_replace()
$phrase = "The sky is blue.";
echo str_replace("blue", "red", $phrase); // Outputs: The sky is red.
```

```
// trim()
$input = "  some user input  ";
$clean_input = trim($input); // $clean_input is "some user input"
```